CUSTOMER CHURN INTELLIGENCE PLATFORM

Complete Technical & Business Analytics Report

Author	Abhishek
Project	Customer Churn Intelligence Platform
Language	Python 3.x
Key Libraries	pandas, scikit-learn, numpy, xlsxwriter, tkinter
Purpose	End-to-end ML-powered churn analytics with automated Excel reporting
Output	Executive Excel report - 6 analytical sheets + 6 interactive charts
Audience	Business stakeholders, data analysts, ML learners, developers

Table of Contents

Table of Contents 2
1. Executive Summary 4
1.1 What Problem Does This Project Solve? 4
1.2 Platform Capabilities at a Glance 4
2. Business Analytics Concepts Explained 6
2.1 Customer Churn 6
2.2 Churn Rate vs Retention Rate 6
2.3 Customer Lifetime Value (CLV) 6
2.4 Revenue at Risk 7
2.5 Customer Segmentation 7
2.6 Churn Probability Score 8
2.7 Feature Engineering: The 4 Derived Metrics 8
2.8 KPI Cards — The Executive Dashboard Metrics 8
3. Machine Learning: Concepts & Implementation 10
3.1 What is Supervised Learning? 10
3.2 Random Forest — The Primary Model 10
3.3 Gradient Boosting — The Fallback Model 10
3.4 Train-Test Split 11
3.5 Model Evaluation Metrics 11
3.6 Heuristic Fallback Score 12
3.7 Synthetic Churn Labels — The Percentile Risk Formula 12
4. Data Pipeline: From Raw File to Insights 13
4.1 File Loading — Multi-Engine Ingestion 13
4.2 Basic Cleaning 13
4.3 Smart Column Detection — find_col() 13
4.4 Numeric Conversion — clean_num() 14
4.5 Churn Label Mapping 14
4.6 Data Consistency Fix — TotalCharges 15
4.7 Label Encoding for Categorical Columns 15
4.8 Median Imputation — SimpleImputer 15
5. Code Architecture & Design 16
5.1 High-Level Structure 16
5.2 Key Functions Reference 16
5.3 The GUI Layer — Tkinter 17

5.4 The Safe Save System 18
5.5 Constants and Configuration 18
6. The Excel Report — Sheet-by-Sheet Breakdown 19
6.1 Sheet 1: Executive Dashboard 19
6.2 Sheet 2: Segment Summary 19
6.3 Sheet 3: High Risk Customers 20
6.4 Sheet 4: Processed Data 20
6.5 Sheet 5: Raw Data 20
6.6 Sheet 6: Data Quality & Pipeline Report 20
6.7 Chart-Source Sheets (7 Hidden Sheets) 20
7. Libraries & Dependencies 22
8. Error Handling & Robustness 23
8.1 Top-Level Exception Handler 23
8.2 Graceful Degradation 23
8.3 Data Quality Documentation 24
9. Excel Formatting System 25
9.1 The MF() Format Factory 25
9.2 Key Format Codes 25
9.3 Alternating Row Colours (Striping) 25
9.4 Conditional Cell Formatting (Risk Highlighting) 25
10. All Formulas & Calculations 27
11. Limitations & Potential Extensions 28
11.1 Current Limitations 28
11.2 Potential Extensions for Learners 28
12. Complete Glossary 29
13. Quick Start Guide 31
13.1 Prerequisites 31
13.2 Running the Application 31
13.3 Required Data Format 31

1. Executive Summary

The Customer Churn Intelligence Platform is a fully automated, end-to-end Python application that ingests raw customer data in multiple formats, applies machine learning models, and produces a professionally formatted Excel executive report — all with a simple graphical interface requiring zero coding knowledge from the end user.

This document is a complete learning guide covering every aspect of the project: the business problem it solves, the data analytics and machine learning techniques employed, the software architecture and design decisions, and an explanation of every major code block. By the end of this report, a learner with basic Python knowledge will be able to fully understand, explain, and extend this platform.

1.1 What Problem Does This Project Solve?

Customer churn — the loss of customers who stop using a product or service — is one of the most critical challenges in business. Studies consistently show that:

- Acquiring a new customer costs 5–7× more than retaining an existing one
- Even a 5% improvement in customer retention can increase profitability by 25–95%
- Businesses that can predict churn in advance can take proactive retention actions (discounts, outreach, loyalty perks) before the customer leaves

This platform automates the entire churn analysis pipeline — from raw data to board-ready insights — making ML-powered analytics accessible to non-technical business users.

1.2 Platform Capabilities at a Glance

Capability	Description
Multi-Format Ingestion	Reads CSV, Excel (.xlsx/.xls/.xlsm/.xlsb), ODS, TSV — 6 encodings, 6 separators tried automatically
Intelligent Detection	Auto-detects Churn, Tenure, Revenue, and ID columns using keyword matching — no manual mapping required
Data Cleaning	Removes blanks, duplicates, unnamed columns; handles currency symbols, European decimal formats, Indian lakh notation
Feature Engineering	Creates 4 new analytical features: AvgMonthlySpend, ValueScore, LoyaltyScore, SpendVariance
ML Churn Prediction	Trains Random Forest (primary) or Gradient Boosting (fallback); produces churn probability per customer
Customer Segmentation	2×2 Risk × Value matrix: High/Low Risk crossed with High/Low CLV — 4 distinct segments
CLV Prediction	Retention-weighted CLV proxy: TotalCharges × (1 − Churn_Prob)

Revenue at Risk	Annualised run-rate for high-risk customers: MonthlyCharges × 12
Synthetic Label Fallback	When no churn column exists, generates probabilistic churn labels using a weighted percentile formula
Excel Report Generation	Produces 13-sheet, 6-chart executive workbook with KPIs, colour-coded tables, and audit trail
GUI Interface	Tkinter dialogs — file picker, save dialog, info/error popups — no terminal required
Safe Save	3-retry logic with auto-rename for locked Excel files

2. Business Analytics Concepts Explained

This section explains every business analytics term used in the project. Understanding these concepts is essential for interpreting the outputs the platform produces.

2.1 Customer Churn

Definition

Customer Churn (also called Customer Attrition) is the percentage of customers who stop doing business with a company during a given time period.

Formula: Churn Rate = (Customers Lost ÷ Total Customers at Start) × 100

Example: If a telecom company starts January with 10,000 subscribers and 400 cancel by month-end, the monthly churn rate = (400 ÷ 10,000) × 100 = 4%

In this platform, churn is represented as a binary label:

- 1 = Churned (customer has left or is predicted to leave)
- 0 = Retained (customer is still active)

The platform supports multiple churn label formats in source data — 'Yes/No', 'True/False', '1/0', 'Active/Inactive', 'Churned/Retained', and many more — all automatically mapped.

2.2 Churn Rate vs Retention Rate

Churn Rate	Percentage of customers lost. Higher is worse. Formula: Churned ÷ Total × 100
Retention Rate	Percentage of customers kept. Higher is better. Formula: 100 − Churn Rate
Industry Benchmark	SaaS: 5-7% annual is excellent. Telecom: 1-2% monthly typical. Retail banking: <1% monthly target
Why it matters	A 2% monthly churn = 22% annual customer loss — company must replace 1 in 5 customers every year just to stay flat

2.3 Customer Lifetime Value (CLV)

Customer Lifetime Value is the total net revenue a business can expect from a customer over the entire duration of their relationship. It is one of the most important metrics in customer analytics.

CLV in This Platform (Simplified Proxy)

True CLV requires survival models or discounted cash flow analysis (complex and data-hungry).

This platform uses a retention-weighted historical proxy:

CLV_proxy = TotalCharges × (1 − Churn_Probability)

Interpretation: The portion of a customer's historical spending expected to be retained going forward.
Use case: Relative ranking — who is most valuable AND most at risk?
Limitation: Not a forward-looking financial CLV. Explicitly documented in the audit sheet.

CLV drives segmentation — customers are classified as High Value (CLV ≥ median) or Low Value (CLV < median), which then combines with churn risk to form the 4 customer segments.

2.4 Revenue at Risk

Definition	The future revenue a business stands to lose if high-risk customers churn
Formula (Platform)	Annualised Risk Revenue = Σ (MonthlyCharges × 12) for all customers with Churn_Prob ≥ 60%
Why annualised?	Monthly charges are the recurring run-rate. Multiplying by 12 converts to annual impact for P&L planning
Historical variant	The platform also shows historical TotalCharges at risk (backward-looking, for context)
Business action	Revenue at risk quantifies the ROI of retention campaigns — if a campaign costs Rs.50,000 but protects Rs.5,00,000 in revenue, the case is clear

2.5 Customer Segmentation

Segmentation is the process of grouping customers with similar characteristics so that different business strategies can be applied to each group. This platform uses a 2×2 Risk × Value matrix:

Customer Risk Level	LOW CLV	HIGH CLV
HIGH RISK (≥60%)	**High Risk – Low Value ⚠ Monitor cost of retention**	**High Risk – High Value □ URGENT: immediate action**
LOW RISK (<60%)	Low Risk – Low Value ✓ Routine engagement	**Low Risk – High Value ★ Loyalty programmes**

Business actions by segment:

- High Risk – High Value: Immediate personal outreach, executive escalation, bespoke retention offers
- High Risk – Low Value: Assess whether retention cost is worthwhile; standard win-back offers
- Low Risk – High Value: Loyalty programmes, VIP treatment, upsell opportunities
- Low Risk – Low Value: Automated engagement, cost-efficient nurturing

2.6 Churn Probability Score

The churn probability score (Churn_Prob, range 0.0 to 1.0) represents the model's confidence that a specific customer will churn. It is produced by the ML model's predict_proba() function.

Score Range	Business Interpretation
0.00 – 0.29	Low Risk — customer likely to stay. Routine engagement sufficient
0.30 – 0.59	Medium Risk — monitor closely. Proactive check-in recommended
0.60 – 0.79	High Risk — retention action required. Flag for outreach this week
0.80 – 1.00	Critical Risk — imminent churn. Immediate escalation and personalised offer

2.7 Feature Engineering: The 4 Derived Metrics

Feature engineering is the process of creating new variables (features) from raw data that better capture business patterns for machine learning. This platform engineers 4 key features:

AvgMonthlySpend	TotalCharges ÷ (tenure + 1). Average spend per month of relationship. Reveals spend trends — a customer paying more early and less now is at higher risk.
ValueScore	log(1 + TotalCharges). Logarithmic transformation of total revenue. Compresses extreme values and makes the distribution more normal — important for ML model stability.
LoyaltyScore	tenure × (TotalCharges ÷ (MonthlyCharges + 1)). Composite score rewarding long tenure AND consistent spend. High score = low churn risk historically.
SpendVariance	|(MonthlyCharges − AvgMonthlySpend)| ÷ (AvgMonthlySpend + 1). Measures how much a customer's current monthly charge deviates from their historical average. High variance often signals upcoming churn.

2.8 KPI Cards — The Executive Dashboard Metrics

Key Performance Indicators (KPIs) are the headline numbers that executives monitor to assess business health. The dashboard presents 14 KPI cards across two rows:

KPI Name	Good/Bad	Business Meaning
Total Customers	Neutral	Size of the customer base being analysed
Churn Rate	Lower=Better	% of customers predicted or confirmed to have churned
Retention Rate	Higher=Better	100 − Churn Rate; % of customers staying
High Risk (≥60%)	Lower=Better	Count and % of customers at imminent churn risk
Medium Risk (30-60%)	Lower=Better	Count of customers requiring monitoring
Low Risk (<30%)	Higher=Better	Count of stable, likely-to-retain customers
Avg Tenure (months)	Higher=Better	Average customer age in months — proxy for loyalty
Avg Customer CLV	Higher=Better	Average predicted lifetime value per customer
Annualised Risk Revenue	Lower=Better	Annual revenue threatened by high-risk churn

Total CLV Portfolio	Higher=Better	Sum of all predicted CLV — the business's value at stake
Avg Monthly Charge	Context	Average revenue per customer per month
ML Model	Info	Which algorithm was used (Random Forest or Gradient Boosting)
Model Accuracy	Higher=Better	% of churn predictions correct on held-out test set
AUC-ROC Score	Higher=Better	Model's ability to rank churners above non-churners (1.0 = perfect)

3. Machine Learning: Concepts & Implementation

3.1 What is Supervised Learning?

Supervised learning is the branch of machine learning where a model learns from labelled examples — data points where the correct answer (label) is already known. The model finds patterns connecting input features to the output label, then applies those patterns to new, unseen data.

In this project	Features = tenure, monthly charges, spend variance etc. Label = Churn (0 or 1). Task = Binary Classification

3.2 Random Forest — The Primary Model

Random Forest is an ensemble algorithm that builds many decision trees and combines their predictions. It is the first model attempted in this platform due to its robustness and interpretability.

n_estimators=300	Builds 300 individual decision trees. More trees = more stable predictions, but slower training
max_depth=12	Each tree can split data up to 12 times. Prevents overfitting (memorising training data)
class_weight='balanced'	Automatically adjusts for imbalanced classes (e.g., if only 10% of customers churned, the model would otherwise predict 'no churn' for everyone and still be 90% 'accurate')
random_state=42	Seeds the random number generator for reproducibility — same result every run
n_jobs=-1	Uses all available CPU cores for parallel training — faster on multi-core machines

3.3 Gradient Boosting — The Fallback Model

Gradient Boosting is another ensemble method that builds trees sequentially — each new tree corrects the errors of the previous one. It often achieves higher accuracy than Random Forest but is slower to train.

n_estimators=150	150 sequential trees. Fewer than RF because each tree is more specialised
max_depth=5	Shallower trees than RF (gradient boosting trees are typically shallow — called 'weak learners')
Sequential building	Unlike RF (parallel), GB builds tree N after seeing tree N-1's errors — this is the 'boosting' concept

Model Selection Logic	Platform tries Random Forest first. If it fails (exception caught), falls back to Gradient Boosting. First successful model is used. Both share the same features and evaluation metrics.

3.4 Train-Test Split

Before training, the dataset is split into a training set and a test set. The model is trained only on the training set and then evaluated on the held-out test set — data the model has never seen.

test_size=0.20	20% of data is reserved for evaluation; 80% used for training
random_state=42	Reproducible split — same data gets the same split every time
stratify=y	Maintains the same churn ratio in both splits — important when churn is rare (e.g., 5% churners). Used only when both classes have enough samples
Minimum class check	Platform checks that the minority class has at least 2 samples in the test set before stratifying — prevents errors with very small datasets

```
# Platform logic for stratification:
min_class = int(y.value_counts().min())
test_size = 0.20
do_stratify = min_class >= max(2, int(len(df) * test_size * 0.10))
Xt, Xe, yt, ye = train_test_split(X, y, test_size=0.20,
                  random_state=42, stratify=(y if do_stratify else None))
```

3.5 Model Evaluation Metrics

Accuracy	Percentage of all predictions that are correct. Formula: (TP + TN) / Total. Misleading when classes are imbalanced — a model predicting 'no churn' always would be 95% 'accurate' if only 5% churn.
AUC-ROC	Area Under the Receiver Operating Characteristic curve. Measures the model's ability to distinguish churners from non-churners across all probability thresholds. 0.5 = random guessing, 1.0 = perfect, >0.8 = very good.
predict_proba()	Unlike predict() which gives hard 0/1 labels, predict_proba() returns the probability of each class. This platform uses the probability of class 1 (churn) as the Churn_Prob score for each customer.

⚠ Synthetic Label Warning — Why Metrics Are Suppressed

When no churn column exists in the input data, the platform generates synthetic churn labels using a risk-score formula.

If ML metrics were shown in this case, they would only measure how well the model learned to replicate the synthetic formula — NOT actual customer churn behaviour.

This would be completely meaningless (and misleading). The platform detects this condition and:

- Displays 'N/A (Synthetic Labels)' for Accuracy and AUC-ROC
- Shows a yellow warning banner on the dashboard
- Documents the suppression reason in the audit sheet

Risk scores and segmentation remain valid for relative ranking purposes.

3.6 Heuristic Fallback Score

When the dataset has fewer than 20 rows, or when all customers have the same churn label (y.nunique() == 1), ML training is impossible. The platform falls back to a heuristic (rule-based) churn probability:

```
# Heuristic: weighted percentile rank combination
heuristic = (
    X['tenure'].rank(pct=True).rsub(1)          * 0.55  # short tenure = higher risk
  + X['MonthlyCharges'].rank(pct=True)          * 0.25  # high charges = higher risk
  + X['SpendVariance'].rank(pct=True)           * 0.20  # high variance = higher risk
).clip(0, 1)
```

This heuristic encodes common business knowledge: new customers with high charges and erratic spending are most likely to churn. It provides a sensible baseline even without ML.

3.7 Synthetic Churn Labels — The Percentile Risk Formula

When the source data contains no churn column, the platform synthesises churn labels using a weighted percentile ranking system. This is explicitly marked as synthetic and documented in the audit trail.

```
# Step 1: Compute spend variance (consistency proxy)
_spend_var = |(MonthlyCharges - AvgMonthlySpend)| / (AvgMonthlySpend + 1)

# Step 2: Weighted risk score (each component is a 0-1 percentile rank)
risk_score = (
    tenure_rank_desc      * 0.40   # short tenure → high risk (40% weight)
  + monthly_charge_rank   * 0.30   # high charges → high risk (30% weight)
  + spend_variance_rank   * 0.20   # high variance → high risk (20% weight)
  + total_charges_rank_desc * 0.10 # low lifetime spend → high risk (10% weight)
)

# Step 3: Top 35% by risk score are labelled as churned
threshold = risk_score.quantile(0.65)
df['Churn'] = (risk_score >= threshold).astype(int)
```

4. Data Pipeline: From Raw File to Insights

The data pipeline is the sequence of transformation steps that converts raw customer data into the cleaned, enriched dataset used for ML and reporting. Each stage is explained below.

4.1 File Loading — Multi-Engine Ingestion

The load_file() function is designed to handle any real-world dataset. Most analytics tools fail on files with unexpected encodings or separators. This platform tries every combination:

Excel engines tried	openpyxl (default), xlrd (legacy .xls), odf (.ods) — with None as final fallback
CSV encodings tried	utf-8-sig, utf-8, latin1, cp1252, iso-8859-1, utf-16 — 6 encodings
CSV separators tried	Auto-detect, comma (,), semicolon (;), tab (\t), pipe (|), colon (:) — 6 separators
Multi-sheet Excel	For Excel files, iterates through all sheets and uses the first one with ≥1 row and ≥2 columns
Error accumulation	All errors are collected and surfaced in a human-readable error dialog if all attempts fail

4.2 Basic Cleaning

Strip column names	str.strip() removes leading/trailing whitespace from all column headers
Drop Unnamed columns	pandas auto-generates 'Unnamed: 0', 'Unnamed: 1' etc. for index columns — these are removed
Drop all-blank rows	dropna(how='all') removes rows where every cell is empty
Drop all-blank columns	dropna(axis=1, how='all') removes columns where every value is missing
Remove duplicates	drop_duplicates() eliminates exact duplicate rows
Reset index	reset_index(drop=True) resets row numbering after deletions

4.3 Smart Column Detection — find_col()

The find_col() function identifies which columns represent churn, tenure, revenue, and customer ID by matching normalised column names against keyword lists. Normalisation removes all non-alphanumeric characters and lowercases everything:

```
def norm(s):
    return re.sub(r'[^a-z0-9]', '', str(s).lower())
```

```
# Example mappings:
# 'Customer ID'    → 'customerid'    → matches 'customerid' keyword ✓
# 'Monthly_Charge' → 'monthlycharge' → matches 'monthlycharge' keyword ✓
# 'Tenure (months)'→ 'tenuremonths'  → matches 'tenure' keyword ✓
```

Keywords are checked in specificity order — most specific first. For example, 'monthlycharge' is checked before 'charge' to avoid falsely matching 'TotalCharges' as the monthly column. Role deduplication ensures no column is used for two different purposes.

4.4 Numeric Conversion — clean_num()

Real-world financial data comes in many formats. The clean_num() function handles all major international number formats:

Currency symbols	₹ $ € £ ¥ — stripped before conversion
US format	1,234.56 — comma as thousands, dot as decimal → becomes 1234.56
European format	1.234,56 — dot as thousands, comma as decimal → detected by regex and corrected
Indian lakh format	1,23,456.78 — detected by regex pattern for Indian number grouping
Whitespace	All internal and surrounding whitespace stripped
Stray characters	Anything not a digit, dot, or minus is removed after currency/locale handling
Coerce to numeric	pd.to_numeric(errors='coerce') converts to float; unparseable values become NaN
40% threshold	Column is converted only if at least 40% of values parse successfully — protects free-text columns from numeric coercion

4.5 Churn Label Mapping

The platform maps over 30 different text representations of churn to binary 0/1 values. The YES and NO frozensets contain every common variation:

```
YES = {'yes', 'y', 'true', '1', '1.0', 'churned', 'left', 'cancelled',
       'cancel', 'exit', 'quit', 'inactive', 'lost', 'gone',
       'departed', 'attrited', 'closed'}

NO  = {'no', 'n', 'false', '0', '0.0', 'active', 'retained', 'stay',
       'stayed', 'current', 'existing', 'ongoing', 'present',
       'alive', 'good', 'loyal'}
```

If text mapping produces fewer than 40% valid labels (null fraction > 0.60), the platform falls back to numeric parsing (treating 1 as churned, 0 as retained). If that also fails, synthetic labels are generated and the source is documented in the audit trail.

4.6 Data Consistency Fix — TotalCharges

A common data quality issue is when TotalCharges is less than MonthlyCharges — logically impossible for any customer who has been with the company for more than one month.

```
# Detect impossible values
bad_total = df['TotalCharges'] < df['MonthlyCharges']

# Fix: impute as tenure × monthly (what it should be)
df.loc[bad_total, 'TotalCharges'] = (
    df.loc[bad_total, 'tenure'] * df.loc[bad_total, 'MonthlyCharges']
)
```

4.7 Label Encoding for Categorical Columns

Machine learning models require numeric input. String columns (like 'Gender', 'Contract Type', 'Payment Method') are encoded to numbers using scikit-learn's LabelEncoder:

```
# Rules:
# 2-20 unique string values → LabelEncoder (e.g., Male=0, Female=1)
# >20 unique values → set to NaN (e.g., free-text comments — useless for ML)
# 0-1 unique values → already uniform, no encoding needed

le = LabelEncoder()
df[col] = le.fit_transform(
    df[col].fillna('__missing__').astype(str)
)
```

Missing values are filled with the string '__missing__' before encoding so the LabelEncoder includes them as a distinct category rather than throwing an error.

4.8 Median Imputation — SimpleImputer

After all transformations, remaining NaN values in the ML feature columns are filled using median imputation:

```
imputer = SimpleImputer(strategy='median')
X = pd.DataFrame(imputer.fit_transform(X_raw), columns=FEATURES)
```

The median is preferred over the mean because it is robust to outliers — one customer with Rs.1,000,000 in charges won't distort the imputed value for everyone else.

5. Code Architecture & Design

5.1 High-Level Structure

The script is a single-file Python application with no class hierarchy — procedural with well-named functions. The top-level flow follows a linear pipeline:

Step	Stage	What Happens
1	**GUI Initialisation**	Tkinter root window created and hidden; always-on-top dialogs configured
2	**Welcome Dialog**	User briefed on capabilities; prompted to select file
3	**File Loading**	load_file() tries every format/encoding/separator combination
4	**Basic Cleaning**	Remove blanks, duplicates, unnamed columns
5	**Column Detection**	find_col() identifies churn/tenure/revenue/ID columns
6	**Numeric Conversion**	clean_num() applied to all non-identity columns
7	**Label Encoding**	Categorical string columns encoded to integers
8	**Core Column Extraction**	pick() resolves tenure, MonthlyCharges, TotalCharges
9	**Consistency Fix**	TotalCharges < MonthlyCharges corrected
10	**Churn Label Resolution**	Map YES/NO → 1/0; or generate synthetic labels
11	**Feature Engineering**	4 derived features computed
12	**ML Training**	Impute → Split → Train RF → fallback GB → predict_proba
13	**CLV & Segmentation**	CLV proxy, 4 segments, Risk_Tier assigned
14	**KPI Computation**	All headline metrics computed
15	**Summary Dialog**	Key findings shown; user prompted for save location
16	**Excel Report Build**	build_report() creates 13-sheet workbook
17	**Safe Save**	safe_write_excel() with retry logic
18	**Success Dialog**	Final summary shown to user

5.2 Key Functions Reference

norm(s)	Normalises a string to lowercase alphanumeric only. Used for column name matching.
clean_num(series)	Converts a mixed-type pandas Series to float, handling all major international number formats.

find_col(df, *kw)	Returns first column whose normalised name contains any of the given keywords, checked in order.
safe_cut(series, bins)	Bins a numeric series into equal-width buckets for chart data. Handles constant/empty series gracefully.
safe_float(v, default)	Coerces any value to a finite float; returns default for NaN, infinity, or errors.
load_file(path)	Multi-engine, multi-encoding file loader. Returns a DataFrame or raises RuntimeError.
pick(col_name, fb_idx, default)	Extracts a numeric series with 3-level fallback: detected column → numeric column by index → constant.
build_report(out_path)	Builds the complete 13-sheet Excel workbook using xlsxwriter.
safe_write_excel(fn, path)	Calls build_report up to 3 times; offers auto-rename or re-pick on PermissionError.
MF(kwargs)**	Format factory — returns xlsxwriter cell formats from keyword arguments.
write_table(ws, df_t, r0)	Writes a striped table with headers to a worksheet at a given row/column offset.
add_chart(type, src, n, title, ws, cell)	Creates and inserts a chart from a named data-source sheet.
data_sheet(name, df_t, title, subtitle)	Creates a hidden chart-source worksheet with title and data.

5.3 The GUI Layer — Tkinter

Tkinter is Python's built-in GUI toolkit. The platform uses it only for dialogs — it never shows a main window (the root is immediately hidden with root.withdraw()).

root = tk.Tk()	Creates the hidden root window — required by Tkinter even if never shown
root.withdraw()	Immediately hides the root window
root.attributes('-topmost', True)	Ensures all dialogs appear on top of other windows (prevents them being hidden behind Excel/browser)
_dlg()	Creates a temporary hidden Toplevel as parent for each dialog — this makes dialogs appear independently without a visible main window
ask_file()	Opens the OS-native file picker dialog (looks like a standard Open dialog on any OS)
save_file()	Opens the OS-native save dialog
info/warn/err()	Show message boxes of different severity levels
ask_yesno()	Shows a Yes/No confirmation dialog; returns True/False

Why Toplevel pattern?	Creating dialogs as children of a Toplevel (rather than root) allows them to appear cleanly without a visible empty Tk() window — a common Tkinter UX trick for script-based applications.

5.4 The Safe Save System

When a user runs this tool repeatedly or if Excel is open, file save conflicts are common. The safe_write_excel() function implements a 3-attempt retry loop with user-controlled resolution:

```
def safe_write_excel(writer_fn, save_path):
    attempt_path = save_path
    for attempt in range(1, MAX_RETRIES + 1):  # 3 attempts
        try:
            writer_fn(attempt_path)             # try to write
            return attempt_path                  # success
        except PermissionError:                 # file locked/open
            # offer: auto-rename OR manual pick
            use_auto = ask_yesno(...)
            if use_auto:
                attempt_path = _timestamped(save_path)  # e.g. Report_143022.xlsx
            else:
                attempt_path = save_file(...)           # user picks new path
        except OSError:
            time.sleep(0.5)  # brief pause before retry
```

5.5 Constants and Configuration

MAX_RETRIES = 3	Number of save attempts before giving up
SHEET_NAME_LEN = 31	xlsxwriter enforces a 31-character limit on worksheet names — _trunc() enforces this
LABEL_ENC_MAX = 20	Categorical columns with more than 20 unique values are dropped (too many categories → poor ML signal)
MIN_ROWS_FOR_ML = 20	Minimum dataset size to attempt ML training — below this, only heuristic scoring is used

6. The Excel Report — Sheet-by-Sheet Breakdown

The output Excel workbook is the primary deliverable. It contains 13 worksheets structured for different audiences — from board-level executives to data analysts to auditors.

6.1 Sheet 1: Executive Dashboard

The flagship sheet. Contains 14 KPI cards across two rows and 6 embedded charts. Designed to be the only sheet an executive needs to review.

Title banner	Full-width merged cell spanning all KPI columns; navy background with white text
Subtitle bar	Grey bar showing file name, model name, accuracy, AUC-ROC, and total customers analysed
Synthetic warning	Yellow warning banner shown if churn labels are synthetic — merged across all columns with high row height (44px)
KPI Row 1 (7 cards)	Total Customers, Churn Rate, Retention Rate, High Risk, Medium Risk, Low Risk, Avg Tenure
KPI Row 2 (7 cards)	Avg CLV, Risk Revenue, Total CLV Portfolio, Avg Monthly Charge, Model Name, Accuracy, AUC-ROC
6 Charts	Segment bar, Risk tier doughnut, Churn probability distribution, CLV distribution, Tenure distribution, Monthly charges distribution
Chart positioning	Charts anchored dynamically based on whether the warning banner is shown — chart_anchor_row adjusts

6.2 Sheet 2: Segment Summary

One row per customer segment with aggregated statistics. The Churn Rate column is colour-coded: RED (>60%), AMBER (30-60%), GREEN (<30%).

Customers	Count of customers in this segment
Churned	Sum of Churn=1 in this segment
Churn Rate	mean(Churn) — colour-coded for at-a-glance risk reading
Avg CLV (Rs.)	Average predicted CLV for the segment — who is most valuable?
Total CLV (Rs.)	Sum of CLV — total revenue at stake per segment
Avg Tenure (mo)	Average customer age — long-tenure high-risk = especially concerning
Avg Monthly	Average monthly charge — high-charge churners need immediate action

Avg Risk Score	Average Churn_Prob — the model's average confidence for this segment

6.3 Sheet 3: High Risk Customers

The retention team's action list. Filtered to customers with Churn_Prob ≥ 60%, sorted highest risk first, capped at 1,000 rows for performance. Includes autofilter and frozen header row.

Churn_Prob formatting: cells ≥ 0.80 are RED (critical), cells 0.60-0.79 are AMBER (high). This allows the team to immediately identify the most urgent cases without reading numbers.

6.4 Sheet 4: Processed Data

The complete analytical dataset after all transformations — all original rows, plus engineered features, churn labels, probabilities, CLV, segment, and risk tier. Includes autofilter and frozen header row. Format applied per column (percentage for Churn_Prob, decimal for CLV, etc.).

6.5 Sheet 5: Raw Data

The original source data exactly as loaded — no transformations applied. This is the audit anchor: if any output looks wrong, the raw data is here for comparison. Includes autofilter and frozen header row.

6.6 Sheet 6: Data Quality & Pipeline Report

A complete audit trail documenting every decision the platform made. Written for compliance, governance, or troubleshooting purposes. Organised into 5 sections:

- File Information — source file details, original vs cleaned row counts
- Column Detection — which columns were found, which were synthetic
- Machine Learning Model — model used, features, accuracy, AUC-ROC, suppression status
- Methodology Notes — CLV formula caveats, revenue-at-risk definition, segmentation rules
- Output KPIs — all 14 headline metrics documented with their values

This sheet is essential for data governance: any stakeholder can verify exactly how the numbers were produced without reading the code.

6.7 Chart-Source Sheets (7 Hidden Sheets)

xlsxwriter builds charts by referencing data in worksheet cells — it cannot embed data within a chart object. The platform creates 7 lightweight data sheets (hidden from casual viewers) that the charts draw from:

Data – Segments	Count per customer segment (4 rows)
Data – Risk Tiers	Count per risk tier: Low/Medium/High (3 rows)
Data – Churn Prob	Distribution of Churn_Prob in 6 buckets

Data – CLV	Distribution of Predicted_CLV in 6 buckets
Data – Tenure	Distribution of tenure in 6 buckets
Data – Monthly	Distribution of MonthlyCharges in 6 buckets

7. Libraries & Dependencies

Library	Import	Role in This Project
pandas	import pandas as pd	Core data manipulation: DataFrames, groupby, merge, string operations
numpy	import numpy as np	Array operations, nan handling, log/clip/isfinite, np.select for segmentation
scikit-learn	from sklearn...	RandomForestClassifier, GradientBoostingClassifier, train_test_split, SimpleImputer, LabelEncoder, accuracy_score, roc_auc_score
xlsxwriter	via pd.ExcelWriter engine	Excel file creation: worksheets, formats, charts, merged cells, column widths
tkinter	import tkinter as tk	GUI dialogs: file picker, save dialog, message boxes, yes/no prompts
os	import os	File path manipulation: basename, splitext, path joining
re	import re	Regular expressions for column normalisation and number format detection
warnings	import warnings	Suppress sklearn deprecation warnings for cleaner console output
sys	import sys	sys.exit(0) for clean cancellation when user closes dialogs
time	import time	time.strftime for timestamped filenames; time.sleep for retry delays
traceback	import traceback	format_exc() for developer-friendly error messages in error dialogs

8. Error Handling & Robustness

Production-quality tools must handle bad inputs gracefully. This platform implements multiple layers of error handling:

8.1 Top-Level Exception Handler

```
try:
    # ... entire pipeline ...
except SystemExit:
    pass  # clean exit when user cancels — not an error
except Exception as e:
    tb = traceback.format_exc()
    err('Unexpected Error', f'{str(e)}\n\nTraceback (last 1200 chars):\n{tb[-1200:]}')
finally:
    try: root.destroy()  # always clean up Tkinter
    except: pass
```

The finally block ensures Tkinter is always destroyed, preventing zombie processes. SystemExit is caught separately to distinguish intentional user cancellation from actual errors.

8.2 Graceful Degradation

The platform never hard-crashes on bad data. Each stage has a defined fallback:

No churn column	→ Synthetic labels generated with documented formula
Churn column unmappable	→ Numeric fallback; if still bad, synthetic labels
ML training fails	→ Heuristic percentile score used instead
Dataset too small (<20)	→ Skip ML entirely; use heuristic only
TotalCharges inconsistent	→ Imputed as tenure × MonthlyCharges
Numeric conversion fails	→ Column set to NaN, excluded from ML features
File save locked	→ 3-retry loop with auto-rename or manual re-pick option
Chart data empty	→ add_chart() returns early if nrows ≤ 0
safe_float() receives NaN	→ Returns default (0.0) without exception
safe_cut() receives constant series	→ Returns ['All Values', count] fallback table

8.3 Data Quality Documentation

Every compromise made during processing is documented in the Data Quality Report sheet. Users are never left guessing why a number looks unexpected. The churn source string is especially important — it tells the user exactly how the churn column was resolved:

- Detected: 'Churn' — real column found and mapped successfully
- Synthetic (no churn column found) — no column matched any keyword
- Synthetic ('Churn' had 75% unmapped values) — column existed but was unreadable

9. Excel Formatting System

The platform implements a custom format factory (MF) that centralises all cell formatting decisions and a pre-built registry (F) of named formats used throughout the report.

9.1 The MF() Format Factory

```
def MF(bold=False, italic=False, sz=10, fc='#0B1F3A', bg=None,
       align='center', valign='vcenter', wrap=False,
       border=1, bc='#C8D0DC', num_fmt=None, left=False):
    # Returns a dict consumed by wb.add_format()
```

Parameters: bold/italic for text weight, sz for font size (in half-points: 10 = 5pt, 20 = 10pt in xlsxwriter), fc = font colour (hex), bg = background colour, align = horizontal alignment, bc = border colour, num_fmt = Excel number format string.

9.2 Key Format Codes

"#,##0"	Integer with thousands separator: 12345 → 12,345
"#,##0.00"	2 decimal places with thousands: 1234.5 → 1,234.50
"0.0000"	4 decimal places (for probability scores): 0.7823
"0.00%"	Percentage from decimal: 0.782 → 78.20%

9.3 Alternating Row Colours (Striping)

Every table in the report uses alternating row colours (white/light grey) for readability. The pattern is:

```
alt = (ri % 2 == 1)  # True for odd rows (0-indexed)
fmt = F['cell_alt' if alt else 'cell']  # light grey vs white
```

Column-specific formats also have _alt variants (e.g., 'num' for white rows, 'num_alt' for grey rows) to maintain the striping while applying numeric formatting.

9.4 Conditional Cell Formatting (Risk Highlighting)

The platform implements manual conditional formatting by passing callable functions as column formats. For example, the cr_fmt function in the Segment Summary sheet:

```
def cr_fmt(val, alt):
    v = float(val)
    if v > 0.6: return F['c_red']    # red background, red text, bold
```

```
    if v > 0.3: return F['c_amber']  # amber background, amber text, bold
    return F['c_green']               # green background, green text, bold
```

10. All Formulas & Calculations

This section consolidates every formula used in the platform with mathematical notation and business interpretation.

Metric	Formula	Business Interpretation
AvgMonthlySpend	TotalCharges ÷ (tenure + 1)	Average revenue per month; +1 prevents division by zero for new customers
ValueScore	log(1 + TotalCharges)	Log-compressed total revenue; compresses outliers for ML stability
LoyaltyScore	tenure × (TotalCharges ÷ (MonthlyCharges + 1))	Rewards both longevity and consistent spend; high score = stable customer
SpendVariance	|MonthlyCharges − AvgMonthlySpend| ÷ (AvgMonthlySpend+1)	Spend instability; sudden price hike or drop often precedes churn
CLV Proxy	TotalCharges × (1 − Churn_Prob)	Historical spend weighted by retention probability
Annualised Risk Rev	Σ (MonthlyCharges × 12) for Churn_Prob ≥ 0.60	Forward-looking annual revenue at stake from high-risk customers
Churn Rate	sum(Churn) ÷ len(df) × 100	Percentage of customers who have churned or are predicted to churn
Retention Rate	100 − Churn Rate	Complement of churn rate; percentage of stable customers
High Risk %	count(Churn_Prob ≥ 0.60) ÷ total × 100	Proportion of customer base at critical churn risk
Risk Score (Synth)	tenure↓×0.40 + monthly↑×0.30 + variance↑×0.20 + total↓×0.10	Weighted percentile rank formula for synthetic churn labelling
Synth Threshold	risk_score.quantile(0.65)	65th percentile cutoff → top 35% flagged as churned
Heuristic Prob	tenure↓×0.55 + monthly↑×0.25 + variance↑×0.20	Simplified heuristic when ML cannot be used

11. Limitations & Potential Extensions

11.1 Current Limitations

- Single-file output: the platform produces one Excel workbook; it does not store results to a database or expose an API
- CLV is a proxy, not a true lifetime value: it uses historical spend adjusted for retention probability — not a survival model or discounted cash flow
- No time-series awareness: the model treats all data as a snapshot; it cannot detect seasonality or trend-based churn patterns
- Binary churn only: the model predicts churned/not-churned; it does not predict when churn will occur (time-to-churn)
- Local execution only: requires Python installation; not a web app or cloud service
- Feature set is fixed: only 7 features used; domain-specific features (number of support calls, product usage metrics) not incorporated
- Label encoding limit: columns with >20 unique string values are dropped; these could instead be one-hot encoded or embedded

11.2 Potential Extensions for Learners

- Add SHAP explainability: show which features most influenced each customer's churn probability (already possible with the trained clf object)
- Add time-series features: rolling 3-month average spend, month-over-month charge change
- Survival analysis: use lifelines library to predict days-until-churn, not just probability
- Web interface: replace Tkinter with Streamlit or Flask for browser-based access
- Database integration: write results to PostgreSQL or SQLite instead of (or in addition to) Excel
- Email alerts: auto-email the High Risk sheet to the retention team when high-risk count exceeds a threshold
- Multi-period analysis: run on monthly snapshots and track how risk scores evolve over time
- Hyperparameter tuning: add GridSearchCV or Optuna to automatically find the best model parameters
- Additional models: add XGBoost, LightGBM, or a simple neural network as additional candidates
- True CLV modelling: implement BG/NBD or Pareto/NBD model for probabilistic CLV estimation

12. Complete Glossary

Alphabetical reference of all technical and business terms used in this project:

AUC-ROC	Area Under the Receiver Operating Characteristic Curve. Measures binary classifier quality. 0.5 = random, 1.0 = perfect.
Churn	When a customer stops using a product or service. The binary label (0/1) this project predicts.
CLV	Customer Lifetime Value. Total net revenue expected from a customer over their entire relationship.
Class Weight	Adjustment applied during model training to compensate for imbalanced class distributions.
Decision Tree	A flowchart-like model that makes predictions by asking a sequence of yes/no questions about features.
Ensemble	A model built by combining multiple weaker models (e.g., many decision trees in Random Forest).
Feature	An input variable used by a machine learning model to make predictions.
Feature Engineering	Creating new, more informative variables from existing raw data.
Gradient Boosting	Ensemble method that builds trees sequentially, each correcting previous errors.
Heuristic	A rule-based approach based on domain knowledge; used when ML is not possible.
Imputation	Filling missing values with a substitute (median, mean, or most frequent value).
KPI	Key Performance Indicator. A headline metric that summarises business performance.
Label	The output variable a model is trained to predict (here: Churn, 0 or 1).
Label Encoding	Converting categorical text values to integers (e.g., Male=0, Female=1).
Median Imputation	Filling NaN values with the column's median. Robust to outliers.
Overfitting	When a model memorises training data instead of learning generalisable patterns.
Percentile Rank	A value's position within a distribution, expressed as 0–1. Used in heuristic scoring.
predict_proba()	Scikit-learn method returning class probabilities instead of hard 0/1 predictions.
Random Forest	Ensemble of decision trees, each trained on a random data and feature subset.
Retention Rate	Percentage of customers retained = 100 − Churn Rate.
Revenue at Risk	Annualised revenue that would be lost if high-risk customers churn.
Risk Tier	Categorical bucket: Low (<30%), Medium (30-60%), High (≥60%) based on Churn_Prob.

Segmentation	Grouping customers by shared characteristics to enable targeted business strategies.
SpendVariance	Measure of how much a customer's current monthly charge deviates from their historical average.
Stratification	Ensuring train/test splits maintain the same class ratio as the full dataset.
Synthetic Labels	Machine-generated churn labels created when no real churn column is available.
Train-Test Split	Dividing data into training (model learning) and test (model evaluation) sets.
xlsxwriter	Python library for creating Excel (.xlsx) files with charts, formats, and formulas.

13. Quick Start Guide

13.1 Prerequisites

```
# Install Python 3.8+
# Then install required libraries:
pip install pandas numpy scikit-learn openpyxl xlsxwriter

# Optional (for .xls and .ods support):
pip install xlrd odfpy
```

13.2 Running the Application

```
# Navigate to the project folder
cd /path/to/project

# Run the script
python churn_analytics.py

# The GUI will guide you through:
# Step 1: Welcome dialog → click OK
# Step 2: File picker → select your CSV/Excel/ODS file
# Step 3: Loading dialog (auto-closes when done)
# Step 4: Summary dialog → review key findings → click OK
# Step 5: Save dialog → choose output location
# Step 6: Success dialog → report is ready
```

13.3 Required Data Format

Your input file should be tabular data with headers. The platform auto-detects columns, but the analysis is richer when these columns are present:

Churn column	Any name containing 'churn', 'attrition', 'left', 'status', etc. Values like Yes/No, 1/0, True/False, Active/Inactive — all supported.
Tenure column	Any name containing 'tenure', 'months', 'duration', 'period'. Numeric values representing months with the company.
Monthly Charges	Any name containing 'monthly', 'charge', 'fee', 'price', 'rate', 'subscription'. Current monthly billing amount.
Total Charges	Any name containing 'total', 'revenue', 'spend', 'bill', 'amount'. Cumulative charges since joining.
Customer ID	Any name containing 'id', 'customerid', 'custid', 'accountid'. Included in High Risk sheet for action.

Note for Learners

Even if your data has none of these columns by name, the platform will still work.
It will use the first numeric columns as fallbacks for tenure, monthly, and total charges.
If no churn column is found, it generates synthetic labels based on spending patterns.

The Data Quality Report sheet in the output Excel will tell you exactly which columns were detected, which were synthesised, and how the model performed.

END OF REPORT

Customer Churn Intelligence Platform • Author: Abhishek • Final Version